Exception Handling LO3

20-2-26

Aim – Write a program to take 2 numbers as input and perform division while demonstrating exception handling with zero and wrong inputs.

Demonstrate the use of a python debugger with intentional errors on a sample debugger

Theory – Exception handling in Python is used to manage runtime errors without stopping program execution. The try block contains code that may cause errors, while the except block handles specific exceptions like ZeroDivisionError, ValueError, IndexError, KeyError, and FileNotFoundError. Custom exceptions allow programmers to define and handle application-specific errors.

A1.

```python
try:
    num1 = input("Enter the first number: ")
    num2 = input("Enter the second number: ")
    num1 = float(num1)
    num2 = float(num2)
    result = num1 / num2
    print(f"Result of {num1} / {num2} = {result}")


except ZeroDivisionError:
    print("Error: Cannot divide by zero! Please enter a non-zero divisor.")
except ValueError:
    print("Error: Invalid input! Please enter valid numeric values.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
else:
    print("Division completed successfully!")
finally:
    print("Program execution finished.")
```

```
Enter the first number: 10
Enter the second number: 0
Error: Cannot divide by zero! Please enter a non-zero divisor.
Program execution finished.
```

```
Enter the first number: 10
Enter the second number: 2
Result of 10.0 / 2.0 = 5.0
Division completed successfully!
Program execution finished.
```

```
Enter the first number: AA
Enter the second number: 12
Error: Invalid input! Please enter valid numeric values.
Program execution finished.
```

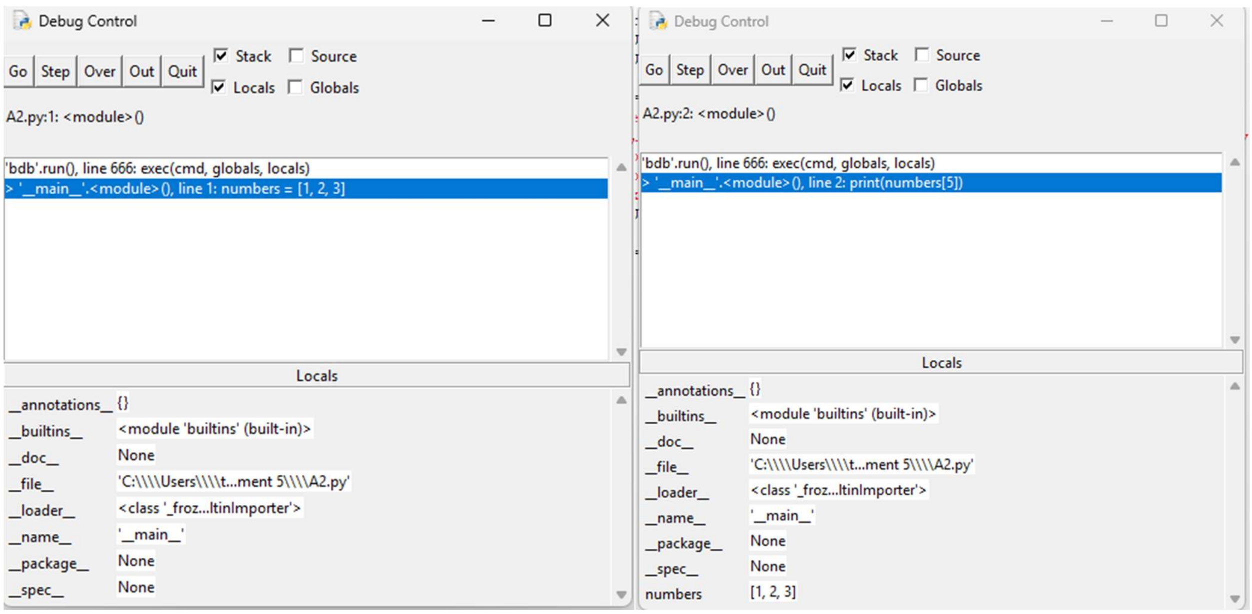
A2.

numbers = [1, 2, 3]

print(numbers[5])

result = 10/0

print(result)



Conclusion

This program demonstrates exception handling and its various use cases. Exception handling improves program reliability by preventing crashes and allowing developers to manage different runtime errors in a controlled way.